

14 - Neural Networks and Backpropagation

Artificial Intelligence Lab Manual - Clean Study Edition

Learning Objectives

- Describe neurons, layers, weights, biases, and activation functions.
- Perform a forward pass through a small multilayer network.
- Explain how backpropagation computes gradients and how gradient descent updates parameters.

Background

A multilayer neural network learns nonlinear mappings by composing linear transformations with nonlinear activations. Backpropagation is an efficient application of the chain rule that provides gradients for every trainable weight.

Core Concepts

Concept	Meaning in this lab
Neuron	Computes a weighted sum followed by an activation.
Activation function	Introduces nonlinearity; examples include ReLU and sigmoid.
Forward pass	Computes predictions from current parameters.
Loss	Quantifies prediction error.
Backpropagation	Applies the chain rule from output toward earlier layers to compute gradients.

Guided Procedure

1. Create a small normalized classification dataset.
2. Define an input layer, one hidden layer, and an output layer.
3. Implement the forward pass using matrix multiplication and activations.
4. Compute a simple loss and derive or inspect output gradients.
5. Update weights with gradient descent and plot loss over training iterations.

Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

rng = np.random.default_rng(3)
W1 = rng.normal(0, 0.2, size=(2, 4))
b1 = np.zeros(4)
W2 = rng.normal(0, 0.2, size=(4, 1))
b2 = np.zeros(1)

def forward(X):
    h = np.maximum(0, X @ W1 + b1)
    yhat = sigmoid(h @ W2 + b2)
    return h, yhat
```

Lab Exercises

Task 1. Implement binary cross-entropy and verify that lower loss generally corresponds to better predictions.

Task 2. Implement the backward gradients for the final layer, then the hidden layer.

Task 3. Compare ReLU and sigmoid in the hidden layer.

Task 4. Use `sklearn.neural_network.MLPClassifier` or a small Keras/PyTorch model to validate the same dataset.

Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

Review Questions

1. Why are nonlinear activations necessary between layers?
2. What mathematical rule underlies backpropagation?
3. What can happen when the learning rate is too large?

Completion check: You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.